

GENERAL — 50 QUESTIONS

ReportFlow Project Interview Guide

Architecture · Groups · Workflow · Security · API · Certificates

50 interview questions with clear, detailed answers in simple English.

ReportFlow · Django College Report Management System
Covers architecture, groups, workflow, security, API, certificates, AI & more.

How to use this guide

Read each question like an interviewer might ask it. The answer explains **what** the feature is, **why** we built it that way, and **how** it works in ReportFlow. Practice saying answers in your own words — do not memorize word-for-word.

Sections in this PDF

- **1. Project Overview & Architecture** — 5 questions
 - **2. Authentication, Users & Security** — 6 questions
 - **3. Report Submission & Approval Workflow** — 7 questions
 - **4. Group Projects & Collaboration** — 4 questions
 - **5. Certificates, QR Verification & PDF** — 7 questions
 - **6. API, Services & Background Jobs** — 6 questions
 - **7. Other Modules & Practical Topics** — 15 questions
-

1. Project Overview & Architecture

Q1. What is ReportFlow and what problem does it solve?

ReportFlow is a college project report management system. Students upload PDF or DOCX project reports (individually or in groups), teachers review them using rubrics, and administrators give final approval.

The system replaces messy email chains and spreadsheets with one place for submissions, group collaboration, feedback, deadlines, messaging, audit logs, personalized QR-verifiable certificates, and optional AI-assisted review.

Q2. What is the main tech stack used in this project?

Backend: **Django 5** with server-rendered templates and a **REST API** at `/api/v1/`.

Database: **MySQL**. Background jobs: **Celery** with Redis (or eager mode in development).

Auth: Django sessions for web pages and **JWT** (SimpleJWT) for API clients.

PDF/certificates: **ReportLab** and **qrcode**. Frontend: Bootstrap 5, Terra theme, vanilla JavaScript.

Q3. Explain the layered architecture of ReportFlow.

Presentation layer — Django views, templates, DRF API views.

Application layer — services like ReportService, CertificateService, AuthService (business rules).

Domain layer — entities and interfaces describing core concepts.

Infrastructure layer — repositories, email, PDF, OCR, database helpers.

This separation keeps views thin and puts business logic in services that can be reused by both web and API.

Q4. Why did you choose Django for this project?

Django gives built-in admin, authentication, ORM, forms, and security features out of the box.

It fits a data-heavy workflow app (reports, approvals, users, notifications) where server-side rendering and REST API can coexist in one codebase.

Django's migration system and mature ecosystem (DRF, Celery integration) speed up development for a college SaaS-style product.

Q5. What are the three user roles and what can each role do?

Student — register, create/join project groups, submit PDF or DOCX reports (individual or group), track status, view group member profiles, comment, request extensions/re-evaluation, download personalized certificates.

Teacher — review assigned reports, score rubrics, approve/reject (with final-submission flag), view group member portfolios, use AI insights, approve pending students, comment.

Admin — final approval, assign group teachers, user management, certificate template designer, bulk actions, system settings (individual + group deadlines), audit log, extension queue, analytics.

Each role gets a different dashboard after login via role-based redirects.

2. Authentication, Users & Security

Q6. How does user registration and account approval work?

Students and teachers self-register from the landing page. New accounts are created with `is_active=False`.

Students need approval from their assigned teacher or an admin. Teachers need admin approval.

Until approved, login is blocked with a friendly message. Admins approve users from User Management.

Credentials can be auto-generated for admin-created users and emailed via SMTP.

Q7. Why use a custom User model instead of Django's default User?

ReportFlow needs extra fields: **role** (student/teacher/admin), **department**, **roll_number**, **assigned_teacher**, profile photo, etc.

Using `AUTH_USER_MODEL = 'accounts.User'` from the start avoids painful migrations later.

All foreign keys reference one consistent user table across reports, messages, and notifications.

Q8. How does login work for both web pages and the API?

Web login uses Django session authentication — a cookie keeps the user logged in.

API login returns JWT access + refresh tokens stored in localStorage by `api_client.js`.

DRF uses `OptionalJWTAuthentication` + `SessionAuthentication` — stale Bearer tokens on the login page do not block session login (fixed in `jwt_optional.py`).

After login, users are redirected to the correct dashboard based on role.

Q9. How does ReportFlow show user-friendly error messages?

Central helper: `core/utls/user_messages.py` — `friendly_message()` sanitizes technical errors before toasts or pages.

API defaults use plain English: "Please sign in to continue", "You don't have permission to do that", etc.

Frontend `api_client.js` maps old technical API messages via `sanitizeFriendlyMessage()` before `ReportFlowToast`.

`AppError` subclasses carry safe messages; raw tracebacks never reach students or teachers.

Q10. How do you prevent logged-out users from seeing cached pages with the back button?

A custom middleware adds **Cache-Control: no-store** headers for authenticated pages.

JavaScript `auth_guard.js` checks session on `pageshow`, `popstate`, and `visibilitychange` via `/accounts/session-check/`.

Logout uses a redirect page that clears JWT tokens and replaces browser history.

Together these reduce the risk of seeing private data after logout using back/forward navigation.

Q11. What password security rules does the project enforce?

Minimum length of 7 characters (project requirement for auto-generated college credentials).

Django validators block common passwords and purely numeric passwords.

Passwords are hashed with Django's default PBKDF2 hasher — never stored in plain text.

Password reset uses Django's token-based email flow with Bootstrap-styled forms.

3. Report Submission & Approval Workflow

Q12. Walk through the full report lifecycle from submit to certificate.

- 1) Student uploads PDF or DOCX (individual or group). 2) Teacher reviews with rubric scores and feedback.
 - 3) Teacher approves and may mark **final submission**. 4) Admin gives final approval and marks.
 - 5) If final + fully approved → personalized certificate PDF(s) with QR, email per recipient, celebration modal on dashboard.
- Status: Pending → Approved or Rejected; resubmits create ReportVersion archives.

Q13. What file types can students submit?

Reports accept **PDF** and **DOCX** (migration 0008, constants in apps/reports/constants.py).

PDFs can be previewed inline; DOCX shows a friendly "cannot preview in browser" message with download link.

AI pipeline uses document_extractor.py — PyMuPDF for PDF, zip+xml parsing for DOCX text.

Same validation (size limit, extension) applies to individual and group submissions via API and web form.

Q14. What happens when a student submits a report?

ReportSubmitForm validates the PDF file type and size. A Report record is created linked to the student.

Default rubric is attached, academic year is set, late flag is calculated against SystemSettings deadline.

Activity log records SUBMITTED. Student and teacher get notifications.

Optional Celery task can queue AI analysis of the PDF after submission.

Q15. Explain the two-step approval process.

Step 1 — Teacher: reviews PDF, fills rubric, sets teacher_marks, approves or rejects with reason.

Step 2 — Admin: reviews teacher-approved reports, sets final marks, admin_approved flag, final status.

This mirrors real colleges where faculty recommends and the department/admin office confirms.

Bulk admin approve/reject is supported for many reports at once.

Q16. What is is_final_submission and why is it important?

A checkbox on teacher approval: "This is the final submission — student receives a certificate after admin approval."

Only reports marked final AND fully approved become certificate-eligible.

This prevents certificates from being issued for draft or practice submissions.

The `is_certificate_eligible` property checks: approved + teacher OK + admin OK + final submission.

Q17. How does report rejection and resubmission work?

Teacher or admin can reject with a written reason stored in `rejection_reason`.

Rejected reports unlock for student edits unless locked. Student uploads a new PDF version.

Previous files are archived as `ReportVersion` records for audit/history.

Rejection emails are sent via Celery SMTP tasks; comments sync to the feedback panel.

Q18. What is soft delete and why use it for reports?

Reports are not physically removed — `is_deleted=True` hides them from normal lists.

Admins can restore soft-deleted reports. Activity log records `DELETED` and `RESTORED` actions.

Soft delete protects against accidental data loss and keeps audit history intact.

Students can soft-delete their own non-approved reports; staff have broader access.

4. Group Projects & Collaboration

Q19. How do project groups work in ReportFlow?

Students create a **ProjectGroup** with department mates (min/max from SystemSettings).

Admin assigns a faculty guide (teacher) from Group teachers page (/reports/groups/manage/).

Only one active report per group — DB constraint prevents duplicate submissions.

ProjectGroupService in application/services/project_group_service.py handles create, list, assign_teacher.

Q20. How does group report submission differ from individual?

Student selects submission_type=group and project_group_id on submit form or API.

Report.student is the submitter; assigned_teacher comes from group.assigned_teacher.

All group members are notified; any member can resubmit after rejection.

Group name appears on report detail, certificates, and notifications.

Q21. How can teachers and students view group member profiles?

Report detail has a **Student profiles** button → /reports/<pk>/member-profiles/.

Shows each member's photo, roll number, department, and link to public portfolio.

Public portfolio: /reports/student/<username>/ (student_profile_public.html).

Teachers assigned to the group can view any mate's full profile for review context.

Q22. What does the Project groups page show to students?

My groups and Public groups lists with **comma-separated member names** plus member count.

Template: templates/reports/project_groups.html. Sidebar nav section for students.

Students create groups from /reports/groups/create/ with mate multi-select (project_group_create.js).

API mirror: /api/v1/project-groups/ (list, create, submittable, assign-teacher).

5. Certificates, QR Verification & PDF

Q23. How does the QR certificate feature work?

When eligible, `CertificateService` generates a landscape PDF using `ReportLab` (`certificate_builder.py`).

Each certificate gets a unique verification code in `certificate_verification_code` (submitter) or `certificate_member_codes_json` for other group members.

QR code links to public page: `/certificates/verify/?code=...` (absolute URL via `SITE_BASE_URL`).

Verify page supports manual code entry and QR scan from uploaded image/PDF (`certificate_verify.js`).

Q24. How do personalized certificates work for group projects?

On admin approval, `ensure_all_recipient_codes()` assigns a unique token per group member.

Submitter uses `report.certificate_verification_code`; mates use `certificate_member_codes_json[user_id]`.

Each PDF shows that member's name, roll number, project title, group name, and their own QR.

Certificate email loop sends one personalized PDF attachment per recipient via Celery.

Q25. What is the certificate celebration modal?

After `certificate_generated=True`, students see a confetti modal on the student dashboard.

Model: `CertificateCelebrationAcknowledgment` — tracks who has acknowledged per report.

Modal: `templates/dashboard/partials/certificate_celebration_modal.html` + `certificate_celebration.js`.

Ack POST to `/reports/<pk>/certificate-celebration/ack/` hides modal until next certificate.

Every group member gets their own celebration when they log in (not just the submitter).

Q26. Explain the admin certificate template designer.

Admin dashboard embeds `CertificateTemplateForm` with multi-tab designer (Start, Colors, Typography, etc.).

Upload reference image → analyze endpoint extracts palette, blur check, suggested `design_json`.

`CertificateTemplateService` + `certificate_template_analyzer.py`; preview before save without persisting.

Multiple templates in library; activate one for all new certificates. Missing image files fail gracefully.

Q27. What conditions must be met before a certificate is issued?

Report status must be `Approved`. `teacher_approved` and `admin_approved` must both be `True`.

`is_final_submission` must be `True` (teacher checkbox).

`CertificateService.build_pdf_bytes()` validates eligibility before generating PDF.

issue_certificate_if_eligible() sends email with PDF attachment and in-app notification.

Q28. Why use absolute URLs inside QR codes?

Phone cameras need a full https:// URL to open the verify page, not a relative /path.

SITE_BASE_URL setting builds the link embedded in the QR (e.g. https://college.edu/certificates/verify/?code=...).

Without absolute URLs, scanning would fail outside the app context.

Q29. What libraries generate the certificate PDF and QR image?

ReportLab draws the landscape certificate: borders, grades, chips, signatures area.

qrcode[pil] generates the QR PNG embedded in the PDF.

CertificateContext dataclass holds student name, project title, marks, dates, verify URL.

PDF bytes are returned to browser download or attached to certificate email.

6. API, Services & Background Jobs

Q30. Describe the REST API structure.

Base path: **/api/v1/**. Modules: auth, reports, project-groups, messaging, qa, dashboard, certificates.

API uses standard envelope: { success, message, data, errors, status_code }.

OpenAPI docs at /api/docs/ via drf-spectacular (Swagger UI).

List endpoints support pagination, search, filtering, and ordering.

Q31. How many automated tests does the project have?

About **70 tests** across api/tests/, core/tests/, and infrastructure/ai/tests/.

Includes EndToEndApprovalFlowTests (submit → approve → certificate → celebration), group certificate tests, member profile tests, and UserFriendlyMessageTests.

Run: python manage.py test. APITestBase in api/tests/base.py provides shared fixtures.

Q32. What is the repository pattern used in this project?

Repositories (e.g. ReportRepository, NotificationRepository) encapsulate database queries.

Services call repositories instead of scattering ORM code everywhere.

Benefits: easier testing, consistent filters, one place to change query logic.

Example: apply_search_filters() mirrors ReportFilterForm params for API and web.

Q33. How are background tasks handled with Celery?

Celery tasks: send_certificate_email, send_rejection_email, notify_user, process_report_ai.

tasks/dispatch.py checks CELERY_TASK_ALWAYS_EAGER — in dev tasks run synchronously without Redis.

In production, Redis broker queues tasks so slow work (email, AI) does not block HTTP requests.

Task names like reportflow.send_certificate_email make monitoring easy.

Q34. How do in-app notifications work?

Notification model stores message, link, is_read, user, type=ALERT.

create_in_app_notification() writes directly for synchronous web views.

queue_user_notification() queues Celery task for async delivery in services.

Bell icon in navbar shows unread count; clicking marks read and navigates to linked page.

Q35. What is logged in the activity audit trail?

ActivityLog records user, action type (SUBMITTED, ADMIN_APPROVED, DELETED, etc.), report, detail text, timestamp.

Admins view/filter logs in the Activity Log viewer.

Supports accountability — who approved, rejected, or changed what and when.
Important for college compliance and debugging disputes.

7. Other Modules & Practical Topics

Q36. How does the messaging module work?

Simple internal inbox: users send messages to each other with subject and body.
Inbox and Sent views with sorting and pagination. `mark_read` for unread tracking.
MessagingService and MessageRepository back the API; templates provide compose modal.
Unread count appears in navbar via context processor.

Q37. Explain the Q&A; and visitor ask feature.

Logged-in users ask questions; admins reply from `qa/home.html` queue.
Landing page visitors submit questions via modal — stored as `VisitorQuestion`, emailed on reply.
FAQ model powers accordion on landing page for common answers.
Separates authenticated help desk from public pre-login inquiries.

Q38. What is re-evaluation and how is it handled?

After full approval, students can request re-evaluation if they disagree with marks.
`ReEvaluationRequest` stores reason, status `PENDING/APPROVED/REJECTED`, `updated_marks`.
Admins resolve from report detail; approved requests update `report.marks`.
Notifications inform student and log `REEVAL_REQUESTED / REEVAL_RESOLVED` in activity log.

Q39. What is a deadline extension request?

Students request more time when `submission_deadline` in `SystemSettings` has passed or is near.
`DeadlineExtensionRequest` goes to admin extension queue for approve/reject.
Approved extensions can extend global deadline (demo: +7 days in settings).
Prevents hard failures when legitimate delays occur.

Q40. How does the leaderboard feature work?

Shows top approved reports by marks, filterable by department.
Teachers see assigned students; students see their department ranking.
`LeaderboardService` aggregates `Report` queryset with `marks__isnull=False`.
Motivates students and gives teachers a quick performance snapshot.

Q41. What database does the project use and why MySQL?

MySQL with `utf8mb4` charset for full Unicode support (names, project titles).
Strict SQL mode enabled via `init_command` for safer data integrity.
Common in college/enterprise environments; Django ORM abstracts most raw SQL.

PyMySQL driver connects Django to MySQL on localhost in development settings.

Q42. How would you deploy ReportFlow to production?

Use Gunicorn/uWSGI + Nginx, set `DEBUG=False`, strong `SECRET_KEY`, `ALLOWED_HOSTS`, `SITE_BASE_URL`.

Store SMTP and DB credentials in environment variables, not in code.

Run Celery workers + Redis for async email/AI. Collect static files (`collectstatic`).

Enable HTTPS, regular DB backups, and monitor logs in `logs/reportflow.log`.

Q43. What were the biggest technical challenges in this project?

Coordinating multi-role approval state (`teacher_approved`, `admin_approved`, status flags) without inconsistency.

Group certificates — unique QR per member, personalized PDFs, verify lookup for member codes.

Certificate template designer — image analyzer, missing-file resilience, preview-before-save.

Keeping web views and REST API in sync through shared service layer; friendly user messages everywhere.

Q44. How do you explain ReportFlow in a 30-second elevator pitch?

"ReportFlow is a college platform where students submit project PDFs or Word files — solo or in groups — teachers grade with rubrics and optional AI assist, admins finalize approval, and the system auto-generates personalized QR-verifiable certificates with a celebration moment, plus messaging, audit logs, and group collaboration."

Emphasize: structured workflow, group support, trust via audit trail and verifiable certificates.

Q45. How are uploaded PDF files stored and served?

Django FileField on Report model saves files under `MEDIA_ROOT` (`media/reports/...`).

`MEDIA_URL` serves files in DEBUG mode; production uses Nginx or cloud storage (S3-compatible).

`view_pdf` streams PDF inline with `FileResponse` after permission check (`_can_view_report`).

Only authorized roles (owner, assigned teacher, admin) can open or download files.

Q46. What is the purpose of ReportVersion and version compare?

Each resubmit archives the old PDF as a `ReportVersion` with `version_number` increment.

`version_compare` view lets staff select two versions side-by-side for audit.

Helps teachers see what changed between reject and resubmit.

Supports academic integrity investigations without losing history.

Q47. How does role-based permission work in API views?

Custom DRF permission classes: `IsStudent`, `IsTeacher`, `IsAdmin` in `api/permissions/`.

Each API view sets `permission_classes` — e.g. only admin can bulk approve.

Services double-check role logic for defense in depth (`PermissionAppError`).

Same rules apply in Django views via `@role_required` decorator.

Q48. What is the standard API response envelope and why use it?

Every API response: `{ success: true/false, message, data, errors, status_code }`.

Frontend `api_client.js` expects this format for toast messages and redirects.

Consistent errors help AJAX forms show user-friendly messages without parsing HTML.

Implemented in `core/api_response/APIResponse.py` and `BaseAPIView`.

Q49. What frontend JavaScript modules power the UI?

`api_client.js` — JWT login, AJAX forms, `sanitizeFriendlyMessage` for toasts.

`toast.js` — 5-second notification toasts. **`auth_guard.js`** — session security.

`certificate_designer.js` — template analyze/preview. **`certificate_celebration.js`** — confetti modal ack.

`certificate_verify.js` — QR scan from image/PDF. **`submit_group.js`** / `project_group_create.js` — group flows.

`report_ai.js` — teacher AI insights. **`rf_table.js`** — table search/sort/pagination.

Q50. How does the Terra theme differ from default Bootstrap?

Custom CSS variables: `terra-forest`, `terra-moss`, `terra-clay`, `terra-parchment` palette.

Source Serif 4 for headings, DM Sans for body — warm academic look not generic Bootstrap blue.

`app-card`, `terra-badge`, `terra-panel-accent` components used across dashboard and landing.

Dark mode via `data-bs-theme` attribute synced with `localStorage`.